SmartDoc AI - Intelligent Document Q&A System

Huỳnh Lê Anh Tuấn*, Nguyễn Võ Anh Kỳ*, Lương Mai Hoàng Văn*,
Ngô Kiến Văn*, Nguyễn Thanh Văn*, Hồ Quốc Đạt*
*Khoa Công nghệ Thông tin, Trường Đại học Sài Gòn
Email: {hlatuan0793, kyy07072005, hv15032005, ngokienvan2005, thanhvan13131, hahodat1958}@gmail.com

***Tóm tắt nội dung*—Dự án SmartDoc AI tập trung nghiên cứu và xây dựng hệ thống hỏi đáp tài liệu thông minh dựa trên kiến trúc Retrieval-Augmented Generation (RAG). Trong quá trình thực hiện, nhóm đã tìm hiểu và áp dụng các công nghệ cốt lõi như mô hình ngôn ngữ lớn Qwen2.5, kỹ thuật biểu diễn văn bản đa ngôn ngữ (Multilingual Embedding với MPNet) và cơ sở dữ liệu vector FAISS.**

Bên cạnh đó, pipeline xử lý dữ liệu được thiết kế hoàn chỉnh, bao gồm trích xuất nội dung từ file PDF bằng PDFPlumber, phân đoạn tài liệu (chunking), lập chỉ mục và truy xuất thông tin. Kỹ năng lập trình được củng cố thông qua việc sử dụng framework LangChain để xây dựng hệ thống và Streamlit để phát triển giao diện người dùng. Ngoài ra, kỹ thuật Prompt Engineering được áp dụng nhằm tối ưu hóa khả năng phản hồi, đặc biệt với tiếng Việt.

Kết quả đạt được cho thấy hệ thống có thể vận hành hiệu quả trên môi trường cục bộ, đảm bảo tính bảo mật và cung cấp câu trả lời chính xác dựa trên tài liệu đầu vào.

***Keywords*-RAG; Qwen2.5; FAISS; LangChain; Streamlit; Prompt Engineering; Hỏi đáp tài liệu**

I. Giới thiệu và công trình liên quan

A. Bối cảnh và động lực

Sự gia tăng nhanh chóng của tài liệu điện tử đặt ra thách thức trong việc tìm kiếm và khai thác thông tin chính xác. Các phương pháp dựa trên từ khóa còn hạn chế về khả năng hiểu ngữ cảnh. Kiến trúc Retrieval-Augmented Generation (RAG) ra đời nhằm kết hợp truy xuất dữ liệu với mô hình ngôn ngữ lớn (LLMs) để tạo ra câu trả lời chính xác và phù hợp hơn.

B. Mục tiêu dự án

Dự án hướng đến xây dựng hệ thống hỏi đáp tài liệu dựa trên RAG, bao gồm:

- Xây dựng pipeline từ tải tài liệu đến sinh câu trả lời.
- Áp dụng embedding đa ngôn ngữ và FAISS cho tìm kiếm ngữ nghĩa.
- Triển khai mô hình Qwen2.5:7B trên môi trường local bằng Ollama.
- Phát triển giao diện người dùng bằng Streamlit.

C. Kiến thức tiếp thu

Nhóm đã tiếp cận các nội dung chính:

- Kiến trúc RAG và cơ chế Retriever–Generator.
- Kỹ thuật NLP: chunking, indexing và semantic search.
- Prompt Engineering để tối ưu phản hồi.
- Sử dụng LangChain và Ollama trong triển khai hệ thống.

D. Công trình liên quan

Dự án tham khảo các nghiên cứu tiêu biểu như: kiến trúc RAG của Lewis và cộng sự [6], thư viện FAISS của Johnson và cộng sự [3], mô hình Transformers của Wolf và cộng sự [5], cùng các tài liệu hướng dẫn từ LangChain [1], Streamlit [4] và Ollama [2].

II. Thực nghiệm

A. Thiết lập môi trường và dữ liệu

Hệ thống được triển khai trên môi trường cục bộ (Windows/Linux) với Python 3.8+ và Ollama để chạy mô hình ngôn ngữ lớn. Dữ liệu thử nghiệm bao gồm các tệp PDF đa dạng như tài liệu kỹ thuật và bài báo khoa học nhằm đánh giá khả năng trích xuất và truy xuất ngữ nghĩa.

B. Cấu hình hệ thống

Pipeline xử lý dữ liệu được thiết kế với các tham số chính như sau:

- Chunking:`chunk_size=1000`,`chunk_overlap=100` để đảm bảo ngữ cảnh.
- Embedding: sử dụng mô hình *paraphrase-multilingual-mpnet-base-v2* (768 chiều), hỗ trợ tốt tiếng Việt.
- Vector Database: FAISS để tối ưu tìm kiếm tương đồng.

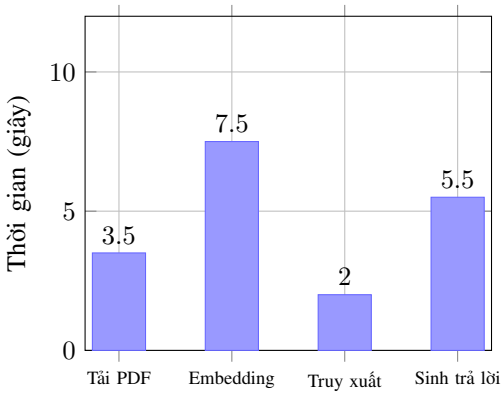
- LLM: Qwen2.5:7B chạy local thông qua Ollama.

C. Hiệu suất hệ thống

Bảng I
THỜI GIAN XỬ LÝ TRUNG BÌNH CỦA HỆ THỐNG

Giai đoạn	Thời gian (giây)
Tải PDF	2–5
Tạo Embedding (100 chunks)	5–10
Truy xuất thông tin	1–3
Sinh câu trả lời (LLM)	3–8

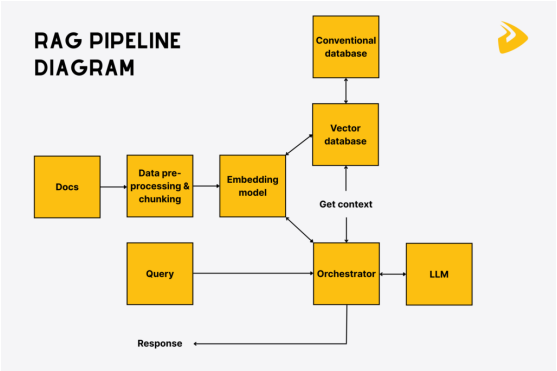
Bảng I cho thấy các tác vụ tạo embedding và sinh câu trả lời chiếm phần lớn thời gian xử lý do liên quan đến tính toán của mô hình AI. Tuy nhiên, tổng thời gian phản hồi vẫn đảm bảo tính tương tác tốt.



Hình 1. Kết quả đo thời gian xử lý các giai đoạn trong hệ thống

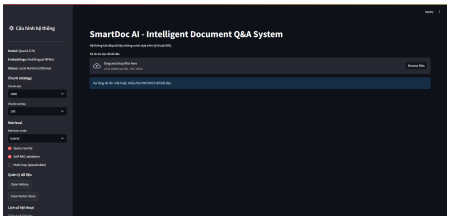
Hình 1 minh họa trực quan sự khác biệt về thời gian giữa các giai đoạn xử lý. Có thể thấy bước tạo embedding và sinh câu trả lời tiêu tốn nhiều tài nguyên nhất, trong khi quá trình truy xuất từ FAISS diễn ra nhanh chóng nhờ tối ưu hóa tìm kiếm vector.

D. Minh họa hệ thống



Hình 2. Kiến trúc tổng thể của hệ thống SmartDoc AI

Như thể hiện trong Hình 2, hệ thống được xây dựng theo kiến trúc RAG bao gồm các thành phần chính: xử lý tài liệu đầu vào, tạo embedding, lưu trữ vector trong FAISS và sử dụng mô hình ngôn ngữ lớn để sinh câu trả lời. Pipeline này đảm bảo việc truy xuất thông tin chính xác và phản hồi theo ngữ cảnh.



Hình 3. Giao diện người dùng của hệ thống trên nền tảng Streamlit

Hình 3 mô tả giao diện người dùng được phát triển bằng Streamlit, cho phép người dùng tải tài liệu PDF và đặt câu hỏi trực tiếp. Kết quả trả lời được hiển thị rõ ràng, hỗ trợ tương tác nhanh và thân thiện.

III. HƯỚNG PHÁT TRIỂN

Trong thời gian tới, dự án sẽ được mở rộng theo các hướng nhằm nâng cao chất lượng và khả năng ứng dụng thực tế của hệ thống.

Trước hết, hệ thống sẽ được đánh giá một cách định lượng thông qua khung RAGAS, cho phép đo lường các tiêu chí như độ trung thực (Faithfulness) và độ liên quan (Relevance) của câu trả lời. Bên cạnh đó, phương pháp LLM-as-a-judge sẽ được áp dụng, sử dụng các mô hình mạnh hơn để đánh giá chất lượng phản hồi.

Tiếp theo, pipeline hiện tại sẽ được nâng cấp thành mô hình Agentic RAG, trong đó hệ thống có khả năng tự đưa ra quyết định như lựa chọn công cụ, định tuyến truy vấn hoặc sử dụng cơ chế lặp để cải thiện kết quả. Việc tích hợp LangGraph sẽ hỗ trợ xây dựng các luồng xử lý linh hoạt hơn.

Ngoài ra, hướng tiếp cận GraphRAG sẽ được nghiên cứu nhằm xây dựng đồ thị tri thức, giúp hệ thống hiểu được mối quan hệ giữa các thực thể trong tài liệu, từ đó cải thiện khả năng trả lời các câu hỏi tổng hợp.

Hệ thống cũng sẽ được mở rộng theo hướng đa phương thức (Multimodal RAG), cho phép xử lý không chỉ văn bản mà còn cả bảng biểu và hình ảnh thông qua các mô hình Vision-Language.

Cuối cùng, việc tối ưu hóa triển khai sẽ được chú trọng nhằm giảm độ trễ và tăng hiệu năng, thông qua các giải pháp như sử dụng engine suy luận hiệu năng cao và áp dụng kỹ thuật nén mô hình (quantization) để vận hành hiệu quả trên môi trường cục bộ.

IV. Tài liệu tham khảo

Tài liệu

[1] LangChain, "LangChain: Building applications with LLMs through composability," [Online]. Available: https://python.langchain.com/

[2] Ollama, "Get up and running with large language models locally," [Online]. Available: https://ollama.com/

[3] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, 2019.

[4] Streamlit, "The fastest way to build and share data apps," [Online]. Available: https://docs.streamlit.io/

[5] T. Wolf *et al.*, "Transformers: State-of-the-art Natural Language Processing," in *Proc. EMNLP*, 2020.

[6] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Proc. NeurIPS*, 2020.